FCND Estimation CPP

Complete Setup Guide — Windows

ArtusGPT / FCND-Estimation-CPP
Visual Studio 2026 Community · Git for Windows · x64

PHASE 1 — INSTALL TOOLS

1 Install Visual Studio Community (free)

Download Visual Studio 2026 Community from:

```
https://visualstudio.microsoft.com/downloads/
```

During installation, on the Workloads screen scroll down to **Desktop & Mobile** and check:

■ **Desktop development with C++**

Leave everything else unchecked. Download is ~7–9 GB and takes 15–30 minutes.

■ If you see 'Package management initialization failed' when opening VS, open the Visual Studio Installer from the Start menu and click Resume or Repair.

✓ Done — VS 2026 Community is installed on your machine.

2 Install Git for Windows

Download the installer from:

```
https://git-scm.com/download/win
```

The site auto-detects Windows and downloads a **.exe** file (e.g. Git-2.45.0-64-bit.exe). Run it and click Next through every screen — all defaults are correct.

After install, open **Git Bash** from the Start menu and verify:

```
git --version
```

✓ Expected: git version 2.x.x.windows.x

■ *Git Bash is a terminal — not an app you open files in. You type commands directly into it.*

PHASE 2 — GET THE CODE

3 Fork the repo on GitHub

Go to the Udacity repo while logged in to your GitHub account:

```
https://github.com/udacity/FCND-Estimation-CPP
```

Click the **Fork** button in the top-right. On the next screen leave all defaults and click **Create fork**.

GitHub creates your personal copy at:

```
https://github.com/ArtusGPT/FCND-Estimation-CPP
```

✓ Done — forked to ArtusGPT/FCND-Estimation-CPP.

4 Clone your fork to your PC

Open **Git Bash** from the Start menu. Navigate to where you want the project. If your Desktop is on OneDrive:

```
cd "C:/Users/kemms/OneDrive/Desktop" # then clone your fork: git clone
https://github.com/ArtusGPT/FCND-Estimation-CPP.git # enter the folder: cd
FCND-Estimation-CPP # list contents to confirm: ls
```

■ If the Desktop path above gives an error, open File Explorer, right-click Desktop in the left panel, click Properties, and use the exact path shown there.

PHASE 3 — BUILD AND RUN

5 Open the project in Visual Studio

In Visual Studio go to **File** → **Open** → **Project/Solution** and open:

```
FCND-Estimation-CPP\project\Simulator.sln
```

A dialog will appear about older toolsets. Click **Retarget all** then **Apply**. The Output panel should say:

```
Retargeting End: 1 completed, 0 failed, 0 skipped
```

■ *This upgrades the project from v141 to your installed v145 toolset. It is normal and expected.*

6 Set platform to x64 and build

In the Visual Studio toolbar near the top, find the platform dropdown. Set it to **x64** (not x86).

Then build: **Build** → **Build Solution** or:

```
Ctrl + Shift + B
```

✓ Expected: 'Build succeeded' with 0 errors in the Output panel.

■ If you see missing header errors, the C++ workload may not have installed. Re-open Visual Studio Installer → Modify and confirm 'Desktop development with C++' is checked.

7 Run the simulator

Press the green ■ button or F5. A 3D simulator window opens with a quadrotor.

✓ Expected: the quad immediately falls to the ground. This is correct — there is no estimator yet so it has no idea where it is.

Press **S** in the simulator to cycle through scenarios. You start at Scenario 06.

IMPORTANT FILE LOCATIONS

These are the only files you edit for the entire project. Everything else is the simulator — do not modify it.

File	Path	Purpose
QuadEstimatorEKF.cpp	src/QuadEstimatorEKF.cpp	Your main EKF implementation — all 6 tasks go here
QuadEstimatorEKF.txt	config/QuadEstimatorEKF.txt	Noise parameters and EKF tuning knobs — edit without restarting
QuadControl.cpp	src/QuadControl.cpp	PID controller — only replaced in the final step (Task 6)
QuadControlParams.txt	config/QuadControlParams.txt	Controller gains — detuned in final step to work with estimator
Scenario configs	config/06_*.txt … 11_*.txt	One file per scenario — read-only, defines sensors and pass criteria

Full Folder Structure

```
FCND-Estimation-CPP/
■■■ src/
■ ■■■ QuadEstimatorEKF.cpp ← EDIT THIS (tasks 1-6)
■ ■■■ QuadControl.cpp ← replace in task 6
■ ■■■ [simulator source — do not touch]
■■■ config/
■ ■■■ QuadEstimatorEKF.txt ← EDIT THIS (noise params)
■ ■■■ QuadControlParams.txt ← edit in task 6
■ ■■■ 06_SensorNoise.txt
■ ■■■ 07_AttitudeEstimation.txt
```

```
■ ■■■ 08_PredictState.txt
■ ■■■ 09_PredictCovariance.txt
■ ■■■ 10_MagUpdate.txt
■ ■■■ 11_GPSUpdate.txt
■■■ lib/ ← Eigen (pre-included, don't touch)
■■■ project/
■■■ Simulator.sln ← open this in Visual Studio
```

Saving Your Work with Git

After you make changes to any file, save them to your GitHub fork with these three commands in Git Bash:

```
# Run from inside the FCND-Estimation-CPP folder git add . git commit -m "Describe what you changed" git push
```

■ *Run these commands regularly — every time you complete a scenario is a good habit. This keeps your work backed up on GitHub.*

The 6 Scenarios (your roadmap)

Scenario	Task	What to implement
06	**Sensor Noise**	Run the simulator to collect GPS and accelerometer data. Calculate standard deviation from the log files in config/log/. Update MeasuredStdDev_GPSPosXY and MeasuredStdDev_AccelXY in config/06_SensorNoise.txt.
07	**Attitude Estimation**	Implement a nonlinear complementary filter using quaternions in UpdateFromIMU(). Goal: attitude error < 0.1 rad for 3+ seconds.
08	**Prediction Step — State**	Implement PredictState() to propagate the state forward using IMU acceleration. The estimated state should track the true state.
09	**Prediction Step — Covariance**	Implement GetRbgPrime() (Jacobian of rotation matrix) and the full Predict() covariance update. The sigma bounds should grow over time.
10	**Magnetometer Update**	Implement UpdateFromMag(). Fuse magnetometer yaw measurement into the EKF. Goal: yaw error < 0.1 rad for 10+ seconds.
11	**GPS Update + Controller**	Implement UpdateFromGPS(). Replace QuadControl.cpp and QuadControlParams.txt with your own controller. Detune gains ~30%. Goal: position error < 1 m for the full flight.

Key Tips

• Eigen is pre-included

The lib/ folder already contains the Eigen matrix library. No separate installation needed. Use it for all vector/matrix operations in your EKF.

- **Read the companion paper**

  Download 'Estimation for Quadrotors' — linked in the repo README. Sections 7.1–7.3 contain exactly the maths for each function you implement.

- **No restart needed for config changes**

  Changes to .txt files in config/ take effect immediately when you save them — you don't need to rebuild or restart the simulator.

- **Rebuild needed for .cpp changes**

  Any change to .cpp files requires Ctrl+Shift+B to rebuild before running the simulator again.

- **GetRbgPrime is the hardest part**

  The Jacobian in scenario 09 is the trickiest step. Triple-check your partial derivatives against section 7.2 of the paper before moving on.

- **Detune the controller in scenario 11**

  Flying from estimated state is harder than ideal state. Reduce position and velocity gains by ~30% in QuadControlParams.txt.
